PRODUCT REQUIREMENTS DOCUMENT

Satoshi's Wall

Bitcoin Layer 1 Permanent Message Board — Built on OPNet

VERSION	DATE	STATUS
1.0.0	February 2026	Live on Testnet

Table of Contents

1 Product Overview

Satoshi's Wall is a permanent, decentralised message board built directly on Bitcoin Layer 1 using OPNet smart contracts. Users pay a small Bitcoin fee to carve a 64-character message into the blockchain — permanently and immutably — with one message allowed per wallet address.

> **CORE VALUE PROPOSITION**
>
> Unlike social media posts that can be deleted, edited, or platform-banned, a message on Satoshi's Wall exists as long as Bitcoin exists — verifiable by anyone, censored by no one.

Attribute	Value
Product Name	Satoshi's Wall
Platform	OPNet (Bitcoin Layer 1 smart contracts)
Network	OPNet Testnet (Signet fork, `opt` bech32 prefix)
Contract Address	`opt1sqqysd49aw7suh5epsvepc8qsy0698apg4s7p02zt`
Wall Capacity	10,000 messages (hard cap)
Post Fee	1,000 satoshis (~$0.01 at current BTC prices)
Message Length	Maximum 64 characters (UTF-8)
Wallet	OPWallet browser extension

2 Problem Statement

2.1 The Impermanence of Digital Expression

Every major platform that hosts human expression — Twitter, Facebook, blogs, forums — is subject to censorship, deletion, business failure, or data loss. The average lifespan of a web page is 75 days. Even "permanent" blockchain projects on Ethereum are subject to chain forks, contract upgrades, and protocol changes.

2.2 The Cost of Permanence

Bitcoin offers the most secure and permanent data layer ever created. However, traditional Bitcoin inscription methods (Ordinals, OP_RETURN) are limited in programmability — you can't enforce rules like "one message per address" or collect fees trustlessly at the Bitcoin level.

2.3 The Gap

There is no product that combines:

- True Bitcoin Layer 1 permanence (not a sidechain or L2)
- Programmable rules (one per wallet, fee enforcement)
- An engaging, consumer-friendly interface
- Direct BTC monetisation without wrapping or bridging

> **THE INSIGHT**
>
> OPNet enables Turing-complete smart contracts executed directly within Bitcoin's consensus layer via Tapscript-encoded calldata. Satoshi's Wall is the first consumer product to leverage this capability for permanent digital expression.

3 Product Vision & Goals

3.1 Vision Statement

"To create the most permanent public record of human expression in history — inscribed directly on the Bitcoin blockchain, accessible to anyone, deletable by no one."

3.2 Primary Goals

Goal	Metric	Status
Deploy functional contract on OPNet testnet	Contract live, messages confirmed on-chain	Done
Enforce one-message-per-wallet rule	Zero duplicate posts in contract state	Done
Collect real BTC fees via contract verification	1,000 sats enforced in `requireMinPayment()`	Done
Consumer-grade UI with real-time updates	Glass morphism frontend, 5s polling	Done
Reach mainnet with 1,000 wall inscriptions	1,000 messages posted	Planned
Fill the wall (10,000 messages)	10,000/10,000 capacity	Future

4 Target Users & Personas

Persona 1 — The Bitcoin Believer

Alex, 32, Software Engineer
"I've been stacking sats since 2017. I want to leave a message for the next generation of Bitcoiners."

Goals: Make a permanent statement about Bitcoin, be an early adopter of OPNet
Pain points: Doesn't want to buy tokens or use complex DeFi. Wants simplicity.
Fee tolerance: Happily pays 1,000 sats (~$1) for permanence

Persona 2 — The NFT Collector / Digital Art Enthusiast

Maya, 26, Designer
"I've minted Ordinals but the fees were insane. I want something affordable and actually permanent."

Goals: Own a unique, verifiable piece of Bitcoin history
Pain points: High gas fees, complicated UX on other chains
Fee tolerance: Familiar with NFT minting costs; 1,000 sats is trivial

Persona 3 — The Crypto Curious

James, 19, University Student
"I've heard of Bitcoin but never done anything on-chain. This seems like an easy first step."

Goals: First on-chain interaction, tell friends
Pain points: Complex wallets, confusing jargon, fear of losing money
Fee tolerance: Very cost-sensitive; 1,000 sats is acceptable

5 Feature Requirements

ID	Feature	Priority	Status
F-01	Post a 64-char message to Bitcoin L1	MUST	Done
F-02	One message per wallet address (enforced on-chain)	MUST	Done
F-03	1,000 sat payment verified in contract	MUST	Done
F-04	Display all wall messages in real time	MUST	Done
F-05	Display total messages and remaining capacity	MUST	Done
F-06	Display total sats raised (treasury)	MUST	Done
F-07	Hall of Fame leaderboard (earliest inscribers)	SHOULD	Done
F-08	Live scrolling feed of recent messages	SHOULD	Done
F-09	Optimistic UI (show pending message immediately)	SHOULD	Done
F-10	OPWallet connect / disconnect	MUST	Done
F-11	Batch message fetching (getMessages)	SHOULD	Done
F-12	Message search / filter	COULD	Planned
F-13	Share message on social media	COULD	Planned
F-14	Mainnet deployment	MUST	Planned

6 Smart Contract Requirements

6.1 Storage Layout

Pointer Slot	Variable	Type	Description
0	`_messageCount`	`StoredU256`	Total confirmed messages
1	`_senders`	`StoredMapU256`	index → sender address (as u256)
2	`_blocks`	`StoredMapU256`	index → Bitcoin block number
3	`_texts1`	`StoredMapU256`	index → message bytes 0–31 (chunk1)
4	`_texts2`	`StoredMapU256`	index → message bytes 32–63 (chunk2)
5	`_hasPosted`	`AddressMemoryMap`	address → (index+1), 0 = not posted
6	`_totalRaised`	`StoredU256`	Cumulative satoshis verified as paid

6.2 Method Specifications

POSTMESSAGE(CHUNK1: UINT256, CHUNK2: UINT256) → BOOL

- MUST verify ≥ 1,000 sat UTXO output to treasury address in `Blockchain.tx.outputs`
- MUST revert if sender has already posted (`_hasPosted[sender] ≠ 0`)
- MUST revert if wall is at capacity (10,000 messages)
- MUST revert if both chunks are zero (empty message)
- MUST store sender, block number, chunk1, chunk2 in respective maps
- MUST increment `_messageCount` and `_totalRaised`

GETMESSAGES(OFFSET: UINT256, LIMIT: UINT256) → BYTES

- Limit is capped at 50 entries per call
- Response format: `uint32 count` followed by `N × 128 bytes`
- Each entry: `sender(32) + blockNumber(32) + chunk1(32) + chunk2(32)`

GETTOTALRAISED() → UINT256

- Returns the cumulative `_totalRaised` storage value in satoshis
- Updated atomically with each successful `postMessage` call

7 Frontend Requirements

7.1 RPC Communication

The frontend MUST use native `fetch()` with the `btc_call` JSON-RPC method directly. The `opnet` library's `JSONRpcProvider` uses `undici` (Node.js HTTP) which fails in browser environments and is intercepted by OPWallet for read calls.

7.2 Polling Strategy

- Poll `getMessageCount()` every 5 seconds (lightweight: 1 call, 32 bytes)
- Only trigger full `getMessages()` fetch when count changes
- After posting: display optimistic "Pending" tile immediately; clear when count increases

7.3 OPWallet Compatibility

The `opnet` library (v≥latest) must be patched to omit `signer` and `mldsaSigner` keys from interaction parameters when they are `null`. OPWallet's `signInteraction` checks for key *presence*, not truthiness, and rejects requests containing these keys even when null.

8 Non-Functional Requirements

Category	Requirement	Target
Performance	Initial page load (JS bundle)	< 200 KB gzipped
Performance	Time to first message display	< 3 seconds on broadband
Reliability	RPC call timeout	15 second abort signal
Reliability	Failed fetches	Keep stale data, no blank wall
Concurrency	Parallel fetch guard	No concurrent full-fetches (mutex ref)
Security	XSS prevention	All message text rendered via React (no dangerouslySetInnerHTML)
Security	Contract input validation	Empty message check, capacity check, duplicate check all on-chain
Accessibility	Colour contrast	WCAG AA for all text elements
Compatibility	Browser support	Chrome/Edge/Firefox latest + OPWallet extension
Responsiveness	Mobile layout	Fully responsive down to 375px viewport

9 Economic Model

9.1 Fee Structure

Fee Type	Amount	Recipient	Enforced By
Post fee	1,000 sats	Treasury wallet	Contract (`requireMinPayment`)
Bitcoin network fee	~5,000–15,000 sats (variable)	Bitcoin miners	OPWallet / Bitcoin mempool

9.2 Revenue Projections

Messages Posted	Treasury Revenue	At $100k BTC
100	100,000 sats	$100
1,000	1,000,000 sats (0.01 BTC)	$1,000
10,000 (full wall)	10,000,000 sats (0.1 BTC)	$10,000

PAYMENT FLOW

The treasury receives sats at the Bitcoin UTXO level — directly into the wallet, with zero smart contract custody. The contract acts as a witness/notary, not a custodian. There is no bridge, wrap, or escrow.

10 Success Metrics

Metric	Target (3 months)	Target (12 months)
Messages posted (testnet)	500	10,000 (full)
Unique wallet addresses	500	10,000
Total sats raised	500,000 sats	10M sats (0.1 BTC)
Daily active users (DAU)	50	500
Page load time	< 2s	< 2s
RPC success rate	> 95%	> 99%

11 Roadmap

Phase 1 — Testnet (Complete)

- AssemblyScript contract with postMessage, getMessages, getTotalRaised, hasPosted

- OPNet testnet deployment
- React frontend with glass morphism UI, OPWallet integration
- Real BTC payment enforcement (1,000 sats to treasury)
- Leaderboard, live feed, optimistic updates

Phase 2 — Mainnet Launch

- Security audit of the smart contract
- Mainnet deployment on OPNet (Bitcoin mainnet)
- Landing page + marketing site
- Social sharing (unique shareable link per message)
- Message search and filtering

Phase 3 — Growth

- Certificate of Inscription NFT (OP721) for each posted message
- Embeddable widget for other websites
- API for third-party integrations
- Mobile-optimised PWA with wallet deep-link support
- Multiple wall categories (art, quotes, dedications)
